UNIVERSIDAD PERUANA DE LOS ANDES

Informe Técnico comparativo sobre arquitecturas centralizadas, cliente-servidor y distribuidas

---

CURSO:

Base de Datos II

DOCENTE:

Mag. Fernandez Bejarano Raul Enrique

ESTUDIANTE:

Rosales Quispe Josmel

CICLO:

V

HUANCAYO - PERÚ

2026-1

1. Introducción

El diseño de la arquitectura de una base de datos es una de las decisiones más críticas en el desarrollo de sistemas de información. La arquitectura elegida determina la escalabilidad, disponibilidad, rendimiento, seguridad y costo total de propiedad de cualquier aplicación moderna.

El presente informe técnico analiza de manera comparativa las tres arquitecturas fundamentales de bases de datos:

- Arquitectura Centralizada
- Arquitectura Cliente-Servidor
- Arquitectura Distribuida

Para cada arquitectura se presentan sus definiciones, características técnicas, ventajas y desventajas, así como ejemplos de implementación con DBMS (Database Management Systems) actuales. Finalmente, se emite una recomendación fundada para un Sistema de Gestión Académica (SGA).

2. Arquitectura Centralizada

2.1 Definición

La arquitectura centralizada es el modelo más sencillo en la gestión de bases de datos. En ella, todos los datos, la lógica de procesamiento y los recursos de almacenamiento residen en un único servidor o mainframe central. Los usuarios acceden a los datos directamente a través de terminales conectadas a dicho servidor.

> 🔑 **Concepto clave**
> Un solo servidor = un único punto de verdad. Todo el poder de cómputo y almacenamiento se concentra en una máquina central que atiende a todos los usuarios.

2.2 Características Técnicas

- Almacenamiento unificado: todos los datos en un único servidor físico o virtual.
- Procesamiento centralizado: la CPU del servidor ejecuta todas las transacciones.
- Gestión simplificada: un solo administrador controla la totalidad del sistema.
- Consistencia alta: modelo ACID nativo sin necesidad de coordinación entre nodos.
- Seguridad perimetral: un solo punto de control de acceso y autenticación.
- Respaldo unificado: estrategias de backup aplicadas a un único repositorio.
- Sin latencia de red: la comunicación interna es local y de muy baja latencia.

2.3 Ventajas y Desventajas

✅ Ventajas	❌ Desventajas
Gestión administrativa muy sencilla	Punto único de fallo (SPOF): si el servidor cae, todo el sistema falla
Bajo costo de implementación inicial	Escalabilidad limitada: solo crece verticalmente (más RAM/CPU)
Consistencia de datos garantizada (ACID)	Cuello de botella: un solo servidor para todos los usuarios
Respaldo y recuperación simples	Sin tolerancia a fallos ni redundancia
Sin complejidad de sincronización entre nodos	Riesgo alto de pérdida de datos ante desastres físicos
Rendimiento óptimo para cargas de trabajo pequeñas	Poco adecuada para entornos geográficamente distribuidos

2.4 Ejemplos de Implementación con DBMS Actuales

- SQLite: motor embebido usado en dispositivos móviles, navegadores y aplicaciones de escritorio. Ideal para datos locales sin servidor.
- Microsoft Access: DBMS de escritorio para pequeñas organizaciones y aplicaciones departamentales.
- Oracle Database Standard Edition: versión centralizada del popular motor empresarial para organizaciones medianas.
- IBM Db2 (modo standalone): solución para mainframes que centraliza procesamiento masivo de datos financieros y estadísticos.

Casos de uso típicos:

- *Aplicaciones de escritorio (inventarios, POS locales).*
- *Sistemas embebidos (dispositivos IoT con SQLite).*
- *Prototipos y entornos de desarrollo.*
- *Instituciones con menos de 200 usuarios concurrentes.*

3. Arquitectura Cliente-Servidor

3.1 Definición

La arquitectura cliente-servidor establece una clara separación de responsabilidades entre dos tipos de componentes: el servidor de base de datos, que gestiona y almacena los datos, y los clientes, que son las aplicaciones o interfaces que solicitan y presentan información a los usuarios. La comunicación se realiza a través de protocolos de red estándar (TCP/IP).

> 🔑 **Concepto clave**
> Separación de responsabilidades: el servidor es el guardián de los datos; los clientes son los consumidores. Esta división permite que múltiples aplicaciones compartan una misma fuente de datos de forma segura y concurrente.

3.2 Características Técnicas

- Arquitectura en capas: presentación (cliente), lógica de negocio (middleware/servidor de aplicaciones) y datos (servidor de BD).
- Protocolo de comunicación: SQL sobre TCP/IP, ODBC, JDBC, o APIs REST/GraphQL.
- Control de concurrencia: gestión de múltiples conexiones simultáneas con bloqueos y transacciones ACID.
- Seguridad basada en roles: usuarios, perfiles y permisos gestionados en el servidor.
- Escalabilidad mixta: vertical en el servidor de BD y horizontal en la capa de aplicación.
- Modelo de tres capas (3-tier): separación de interfaz, lógica y persistencia.

3.3 Ventajas y Desventajas

✅ Ventajas	❌ Desventajas
Separación clara de responsabilidades (MVC/n-tier)	Dependencia de la red: si la red falla, los clientes pierden acceso
Múltiples clientes comparten la misma BD	El servidor sigue siendo un punto de fallo (mitigable con clustering)
Seguridad granular: roles, perfiles y permisos	Costo de licencias para DBMS enterprise
Mantenimiento modular (actualizar cliente sin tocar BD)	Configuración y administración más complejas que la centralizada
Escalabilidad de la capa de aplicación	Latencia variable según la carga de red
Soporte robusto para transacciones ACID y concurrencia	Requiere personal especializado en DBA

3.4 Ejemplos de Implementación con DBMS Actuales

- MySQL 8.x / MariaDB: el DBMS open-source más popular para aplicaciones web (WordPress, Moodle, ERP). Ampliamente usado en LAMP/LEMP.
- PostgreSQL 16: motor relacional avanzado con soporte para JSON, extensiones geoespaciales (PostGIS) y replicación. Preferido por su robustez y conformidad con estándares SQL.
- Microsoft SQL Server 2022: solución empresarial con integración nativa con el ecosistema Microsoft (Azure, Power BI, .NET).
- Oracle Database Enterprise Edition: referente en banca, telecomunicaciones y gobierno por su alto rendimiento y características avanzadas.
- IBM Db2 (modo cliente-servidor): usado en entornos financieros con alta demanda transaccional.

Casos de uso típicos:

- *Sistemas de información empresariales (ERP, CRM, HIS).*
- *Aplicaciones web con alta concurrencia.*
- *Sistemas académicos, hospitalarios y gubernamentales.*
- *E-commerce de mediana escala.*

4. Arquitectura Distribuida

4.1 Definición

En la arquitectura de bases de datos distribuidas, los datos se fragmentan (sharding) y/o replican a través de múltiples nodos independientes que pueden estar físicamente ubicados en distintos

servidores, centros de datos o regiones geográficas. Cada nodo puede procesar consultas localmente, pero el sistema actúa como una única base de datos lógica para las aplicaciones.

> 🔑 **Concepto clave**
> El Teorema CAP (Brewer, 2000) establece que un sistema distribuido solo puede garantizar simultáneamente dos de tres propiedades: Consistencia (C), Disponibilidad (A) y Tolerancia a particiones (P). Los DBMS distribuidos deben elegir sus compromisos (e.g., CP o AP).

4.2 Características Técnicas

- Fragmentación: los datos se dividen en shards (horizontal) o particiones (vertical) distribuidas en múltiples nodos.
- Replicación: copias de datos en múltiples nodos para garantizar disponibilidad y tolerancia a fallos.
- Transparencia de distribución: las aplicaciones ven una única BD lógica sin conocer la distribución física.
- Consistencia eventual: en sistemas AP, los nodos convergen hacia un estado consistente, aunque no de forma inmediata.
- Sin SPOF (Single Point of Failure): la caída de un nodo no detiene el sistema.
- Escalabilidad horizontal: se añaden nodos para incrementar capacidad sin límite teórico.
- Coordinación distribuida: algoritmos de consenso (Raft, Paxos) para elecciones de líder y commits distribuidos.

4.3 Ventajas y Desventajas

✅ Ventajas	❌ Desventajas
Alta disponibilidad: el sistema continúa operando ante fallos de nodos	Alta complejidad operativa y de configuración
Escalabilidad horizontal casi ilimitada (sharding automático)	Consistencia eventual: puede haber lecturas de datos desactualizados
Sin punto único de fallo (alta tolerancia a fallos)	Debugging y monitoreo más difíciles
Distribución geográfica: datos cerca del usuario = baja latencia	Costos operativos altos (infraestructura multi-nodo)
Soporte para Big Data y cargas masivas de lectura/escritura	Transacciones distribuidas complejas (2PC, Saga pattern)
Ideal para microservicios y arquitecturas cloud-native	Requiere equipo con expertise en sistemas distribuidos

4.4 Ejemplos de Implementación con DBMS Actuales

- Apache Cassandra: BD NoSQL orientada a columnas, diseñada para alta disponibilidad sin SPOF. Modelo AP. Usada por Netflix, Instagram.
- MongoDB Atlas: BD documental con sharding automático y replicación global. Ideal para datos semiestructurados y APIs REST.
- Google Cloud Spanner: DBMS NewSQL que combina consistencia ACID global con escalabilidad distribuida. Usado por Google Ads, fintech.
- CockroachDB: BD SQL distribuida con protocolo Raft. Compatible con PostgreSQL. Garantiza consistencia global (CP).

- Amazon DynamoDB: servicio gestionado de BD NoSQL clave-valor con rendimiento de milisegundos. Modelo AP. Usado masivamente en AWS.
- Redis Cluster: almacén de datos en memoria distribuido para caché, sesiones y pub/sub. Latencia sub-milisegundo.

Casos de uso típicos:

- *Redes sociales y aplicaciones con millones de usuarios.*
- *IoT y telemetría en tiempo real.*
- *Plataformas de streaming y contenido (Netflix, Spotify).*
- *E-commerce global con alta disponibilidad 24/7.*
- *Analítica de Big Data e inteligencia de negocios.*

5. Organizador de Conocimientos — Tabla Comparativa General

La siguiente tabla sintetiza los criterios más relevantes para la selección de una arquitectura de base de datos, facilitando la toma de decisiones en función del contexto tecnológico y organizacional:

Criterio	Centralizada	Cliente-Servidor	Distribuida
Definición	Un único servidor central almacena y gestiona todos los datos	Servidor(es) y múltiples clientes con roles diferenciados	Datos fragmentados y replicados en nodos geográficamente dispersos
Escalabilidad	Vertical (limitada)	Horizontal/Vertical	Horizontal (alta)
Disponibilidad	Baja (SPOF)	Media	Alta
Consistencia	Alta (ACID)	Alta (ACID)	Variable (CAP theorem)
Latencia	Baja (local)	Media	Variable (red)
Costo inicial	Bajo-Medio	Medio	Alto
Mantenimiento	Simple	Moderado	Complejo
Seguridad	Centralizada/fácil	Roles y perfiles	Distribuida/compleja
Tolerancia a fallos	Baja	Media	Alta
Ejemplos DBMS	SQLite, MS Access, Oracle SE	MySQL, PostgreSQL, SQL Server	Cassandra, MongoDB Atlas, Google Spanner, CockroachDB

5.1 Comparativo de Ventajas y Desventajas

	Centralizada	Cliente-Servidor	Distribuida
✅ **Ventajas**	• Gestión sencilla• Bajo costo inicial• Alta consistencia• Respaldo unificado	• Arquitectura flexible• Buena escalabilidad• Seguridad por capas• Mantenimiento modular	• Alta disponibilidad• Escalabilidad extrema• Tolerancia a fallos• Sin SPOF
❌ **Desventajas**	• Punto único de fallo• Escalabilidad limitada• Cuello de botella• Sin redundancia	• Dependencia de red• Costo de licencias• Configuración inicial• Latencia variable	• Alta complejidad• Consistencia eventual• Costos operativos• Debugging difícil

6. Recomendación para un Sistema de Gestión Académica (SGA)

6.1 Contexto y Requisitos del SGA

Un Sistema de Gestión Académica (SGA) es una plataforma de misión crítica que debe soportar operaciones concurrentes de docentes, estudiantes, personal administrativo y directivos. Los requisitos fundamentales son:

- Alta disponibilidad durante períodos de matrícula y evaluaciones.
- Consistencia estricta en calificaciones, matrículas y registros oficiales.
- Escalabilidad para soportar picos de carga (inicio de semestre, exámenes finales).
- Seguridad robusta: protección de datos personales (GDPR, Ley de Protección de Datos).
- Capacidad de reportes y analítica institucional.
- Integración con portales web, apps móviles y sistemas externos (SUNEDU, MINEDU).

6.2 Arquitectura Recomendada: Cliente-Servidor como núcleo principal

> ✅ **RECOMENDACIÓN PRINCIPAL**
> Para la mayoría de instituciones educativas (desde pequeñas hasta grandes universidades), la arquitectura Cliente-Servidor es la más adecuada como núcleo del SGA, complementada estratégicamente con componentes distribuidos para analítica y caché. Se recomienda PostgreSQL 16 como DBMS principal.

6.3 Arquitectura Híbrida Recomendada por Módulo

Una arquitectura híbrida inteligente permite aprovechar las fortalezas de cada paradigma según el módulo funcional:

Componente / Módulo	Arquitectura recomendada + DBMS
Datos maestros (alumnos, docentes, cursos)	Cliente-Servidor → PostgreSQL 16
Historial académico y calificaciones	Cliente-Servidor → MySQL 8 / SQL Server 2022
Reportes y analítica institucional	Distribuida → Google Spanner / BigQuery
Sesiones y caché de portal web	Distribuida → Redis Cluster
Documentos y adjuntos (sílabos, tesis)	Distribuida → MongoDB Atlas
Pequeñas instalaciones (<500 usuarios)	Centralizada → SQLite / PostgreSQL local

6.4 Justificación de la Recomendación

- Madurez tecnológica: PostgreSQL y MySQL tienen décadas de desarrollo, amplia comunidad, soporte empresarial y ecosistema de herramientas maduro.
- Consistencia transaccional: las matrículas, calificaciones y récords académicos requieren ACID estricto, propiedad nativa de los DBMS cliente-servidor.
- Costo-beneficio: las licencias open-source (PostgreSQL, MySQL) reducen el costo total, manteniendo rendimiento enterprise.

- Escalabilidad suficiente: para el 95% de instituciones educativas, la escalabilidad vertical + replicación es más que suficiente. Solo las megauniversidades con decenas de miles de usuarios necesitarán distribución total.
- Equipo técnico disponible: hay mayor disponibilidad de profesionales capacitados en SQL y DBMS cliente-servidor que en sistemas distribuidos.
- Integración con sistemas existentes: la mayoría de plataformas académicas (Moodle, SAP, Banner) están diseñadas para trabajar con DBMS relacionales cliente-servidor.

6.5 Cuándo Escalar a una Arquitectura Distribuida

Se recomienda evaluar la migración parcial o total hacia arquitectura distribuida cuando:

- El número de usuarios concurrentes supera los 10,000 activos simultáneamente.
- La institución tiene sedes en múltiples países o regiones con latencia inaceptable.
- Los volúmenes de datos superan los 10 TB y el crecimiento es constante.
- Se requiere disponibilidad 99.99% (Four Nines) para operaciones críticas.
- Se implementan proyectos de analítica avanzada o machine learning sobre datos académicos.

7. Conclusiones

El análisis comparativo de las tres arquitecturas de bases de datos permite extraer las siguientes conclusiones:

- No existe una arquitectura universalmente superior. La elección depende del contexto específico: tamaño de la institución, volumen de datos, presupuesto, necesidades de disponibilidad y capacidades del equipo técnico.
- La arquitectura centralizada es idónea para entornos pequeños, prototipos o sistemas con usuario único, pero resulta inadecuada para sistemas de misión crítica por su inherente punto único de fallo.
- La arquitectura cliente-servidor representa el equilibrio óptimo entre simplicidad, rendimiento, consistencia y costo para la gran mayoría de los sistemas de información empresariales y académicos. Es la opción recomendada para un SGA.
- La arquitectura distribuida ofrece ventajas significativas en escalabilidad y disponibilidad, pero conlleva una complejidad operativa considerable. Su adopción debe ser incremental y justificada por necesidades específicas de escala.
- La tendencia actual apunta hacia arquitecturas híbridas: un núcleo cliente-servidor para datos transaccionales críticos, combinado con componentes distribuidos para caché, analítica y contenido no estructurado.

> 📌 **Conclusión Final**
>
> Para un Sistema de Gestión Académica moderno, se recomienda implementar una arquitectura cliente-servidor con PostgreSQL 16 como DBMS principal, enriquecida con Redis para caché de sesiones y MongoDB Atlas o Google BigQuery para analítica e informes institucionales. Esta combinación ofrece el mejor balance entre robustez, costo, mantenibilidad y capacidad de evolución futura.